DALHOUSIE UNIVERSITY

Data Preprocessing Foundations

Tidy data, missing values, and transformation

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 What data preparation does

What it is for Data preparation turns the table you received into the table an analysis needs. It follows data understanding in the CRISP-DM process [1]. Later findings may send you back to check the original data, so the process is iterative.

The rule Match each problem to an action (Table 1). A unit conversion changes the value and the unit together, for example $C = (F - 32) \times 5/9$ for a Fahrenheit reading.

Table 1. Problems and corresponding preparation steps

Problem	Preparation step
A temperature column mixes units	Transform to a common unit.
9999 is a documented missing code	Clean the representation of missingness.
A total and a count suggest a ratio	Create a derived variable.
Too many variables for a useful analysis	Consider dimensionality reduction.

Table 2 groups these actions into four categories. The categories overlap. Reshaping may expose a cleaning problem, and a new variable may need a transformation.

Table 2. Four categories of data preparation

Cleaning	Interpret types, missing codes, and errors.
Transformation	Change units, scale, or representation.
Feature engineering	Derive useful variables from recorded information.
Dimensionality reduction	Reduce the number of variables used in an analysis.

Check meaning before changing a value Before any cleaning step, record what one row represents, which units and types are expected, and which missing codes are documented. A missing value can remain missing. Filling every gap is not the goal.

2 Tidy data and reshaping

What it is for Most R and Python tools expect a table in which each row is one observation. Reshaping puts the data in that form without changing any value.

Tidy structure

Use one variable per column, one observation per row, and one value per cell. Keep one type of observational unit in each table. Decide what counts as an observation before choosing the layout. The column, row and table rules follow Wickham [4].

The layout follows from the observational unit. A student-level analysis can treat Math grade and English grade as two variables. In the example below, the unit is **one student in one course**, so course name is one variable and grade is another. The same information can need different layouts for different questions.

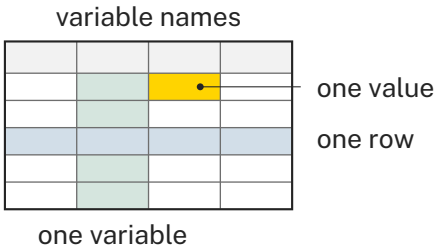


Figure 1. Variables in columns, observations in rows, one value per cell

Worked example, diagnosis Table 3 is the raw grades table. Its data dictionary defines grades on a 0–100 scale and uses 9999 for a missing grade. It defines `DegreeYear` as a degree label followed by an entrance year, separated by a literal pipe character.

Table 3. Raw data, with a documented missing-value code

Name	Math	English	DegreeYear
Luis	90	86	CS|2020
Maria	85	9999	Math|2013
Jose	70	60	CS|2015

Getting one row per student and course takes three decisions.

1. Split `DegreeYear` into two variables, `Degree` and `Year`.
2. Move course names into a `Course` column and scores into a `Grade` column.
3. Replace the documented code 9999 with `NA` (R) or `NaN` (Python).

The first two are reshaping. The third is value cleaning and has a different reason, the data dictionary. An impossible value with no documentation would need investigation first. Keep the raw table so that each step can be checked.

Worked result Table 4 shows the reshaped table. Three student rows become six student-course rows. Maria's missing English grade remains missing. No grade has been invented.

Table 4. One row per student and course after reshaping

Name	Course	Grade	Degree	Year
Luis	Math	90	CS	2020
Maria	Math	85	Math	2013
Jose	Math	70	CS	2015
Luis	English	86	CS	2020
Maria	English	*NA*	Math	2013
Jose	English	60	CS	2015

Setup code The setup creates `grades`, the raw input, and `clean_grades`, a copy with 9999 decoded. Every later example in this handout runs from top to bottom after it in a fresh session. R uses `tidyr`; Python uses pandas and NumPy. Choose either language. The tables and results are the same.

R

```r
grades <- data.frame(
  Name = c("Luis", "Maria", "Jose"),
  Math = c(90, 85, 70),
  English = c(86, 9999, 60),
  DegreeYear = c("CS|2020", "Math|2013",
                 "CS|2015")
)
# The example's data dictionary defines
# 9999 as missing.
clean_grades <- grades
clean_grades$English[
  clean_grades$English == 9999] <- NA
```

Python

```python
import numpy as np
import pandas as pd

grades = pd.DataFrame({
    "Name": ["Luis", "Maria", "Jose"],
    "Math": [90, 85, 70],
    "English": [86, 9999, 60],
    "DegreeYear": ["CS|2020", "Math|2013",
                   "CS|2015"],
})
# The example's data dictionary defines
# 9999 as missing.
clean_grades = grades.copy()
clean_grades["English"] = (
    clean_grades["English"].replace(9999, np.nan)
)
```

Reshape code `separate_wider_delim` (R) and `str.split` (Python) split the combined field, both reading the pipe literally. The year becomes an integer because it is a year, not a label. Then `pivot_longer` (R) or `melt` (Python) moves the two course columns into one grade column. The output has six rows, five columns and one missing grade.

R

```r
tidy_grades <- tidyr::separate_wider_delim(
  clean_grades, DegreeYear, delim = "|",
  names = c("Degree", "Year")
)
tidy_grades$Year <- as.integer(tidy_grades$Year)
tidy_grades <- tidyr::pivot_longer(
  tidy_grades, c(Math, English),
  names_to = "Course", values_to = "Grade",
  cols_vary = "slowest"
)
tidy_grades <- tidy_grades[c(
  "Name", "Course", "Grade", "Degree", "Year"
)]
print(tidy_grades)
```

Python

```python
tidy_grades = clean_grades.copy()
tidy_grades[["Degree", "Year"]] = (
    tidy_grades["DegreeYear"]
    .str.split("|", expand=True, regex=False)
)
tidy_grades["Year"] = (
    tidy_grades["Year"].astype(int))
tidy_grades = tidy_grades.drop(
    columns="DegreeYear"
).melt(id_vars=["Name", "Degree", "Year"],
       var_name="Course", value_name="Grade")
tidy_grades = tidy_grades[[
    "Name", "Course", "Grade", "Degree", "Year"
]]
print(tidy_grades)
```

Differences to know

- R's `cols_vary="slowest"` keeps all Math rows before all English rows, which is the order pandas `melt` produces. Both versions then select the same column order explicitly.
- Older tidyr examples use `gather` and `separate`. The code here uses their current replacements.

3 Dates and strings

What it is for Dates and text labels must be read the same way on every row before they can be grouped, compared or joined.

The rule Parse a date with its stated format. A timestamp also needs a time zone, because a calendar date alone does not identify an instant. Do not treat a local timestamp as UTC without checking

how it was recorded. Normalize the case and surrounding spaces of a label only when the data dictionary says they carry no meaning. Table 5 lists the tools.

Table 5. Date and string tools in R and Python

Task	R	Python equivalent
Handling dates and time zones	`lubridate`	`datetime`, `zoneinfo`, `pandas.to_datetime` with `tz_localize` / `tz_convert`
Handling strings	base R: `grep`, `strsplit`, `nchar`, `substr`, etc.; then `stringr`	`str` methods, the `re` module, the pandas `.str` accessor
More complex string cases	`stringi`	`regex` module, `unicodedata`

Worked example The labels `" North "`, `"north"` and `"SOUTH"` become north, north and south after trimming and lowering case. Two regions remain, not three. The two dates are parsed with the format `%Y-%m-%d`.

R

```r
labels <- c(" North ", "north", "SOUTH")
trimws(tolower(labels))
# "north" "north" "south"
as.Date(c("2026-09-15", "2026-09-16"),
        format = "%Y-%m-%d")
# Timestamps also need a time zone.
```

Python

```python
labels = pd.Series([" North ", "north", "SOUTH"])
print(labels.str.strip().str.lower().tolist())
# ['north', 'north', 'south']
pd.to_datetime(["2026-09-15", "2026-09-16"],
               format="%Y-%m-%d")
# Timestamps also need a time zone.
```

Differences to know

- R offers base string tools, `stringr` and `stringi`, and `lubridate` for dates and times.
- Python offers string methods, regular expressions and Unicode tools, with `datetime`, `zoneinfo` and pandas for time-zone-aware dates.
- Extra packages are options, not a reason to switch language. The same cleaning decisions apply in either.

4 Missing values, three mechanisms

What it is for A gap needs an explanation before it is handled. First record what is missing. Then state how the gap could have arisen, because that assumption decides which handling is defensible.

Assumptions about the recording process

MCAR Missing completely at random. Missingness is independent of both observed and unobserved values.

MAR Missing at random. After accounting for observed information, missingness does not depend on the unobserved values.

MNAR Missing not at random. Missingness still depends on unobserved information after accounting for what was observed.

The three mechanisms follow Rubin [3] and Little and Rubin [2]. These statements describe how gaps arise. They do not say that the values of a variable are themselves random. A missingness percentage cannot determine the mechanism.

Worked examples Table 6 gives one case of each. For the MAR cases, the conditioning information must be recorded. A battery failure is an MCAR case only when its occurrence is independent of the measurements and of the recording order. Otherwise a plausible story can hide systematic missingness.

Table 6. Mechanisms and the assumptions that distinguish them

	Instrument example	Survey example
MCAR	Readings are lost independently of measured values and recording order.	A random subset of forms is lost, independently of responses.
MAR	Failures depend on recorded surface type, not on the reading within a type.	Response depends on recorded delivery method, not on the answer within a method.
MNAR	An unrecorded extreme reading makes failure more likely.	Response depends on the undisclosed answer, even given recorded variables.

What the data can and cannot show You can check whether gaps are associated with observed variables, which may contradict MCAR. The observed data alone cannot distinguish MAR from MNAR. That needs knowledge of the collection process, additional information, or an explicit assumption.

Take a sensor that fails more often outdoors than indoors, with location and readings recorded when available. MAR requires that, once recorded location is accounted for, failure no longer depends on the unobserved reading. If failure also depends on an unrecorded extreme reading, the mechanism is MNAR.

5 Handling gaps by omission

What it is for Omission analyses the observed values and sets incomplete entries aside. It is the simplest response and the first of three (Figure 2).

- **Omission** uses observed data and discards selected incomplete entries or rows.
- **Imputation** supplies estimated values for gaps and keeps a record of which entries were missing.
- **Model-based handling** represents missing values within the analysis model instead of first making one complete table. It is not a Lab 02 task.

None is a universal repair. Choose in relation to the question, the recording process and the information available. Leaving a value missing is sometimes the most defensible decision.

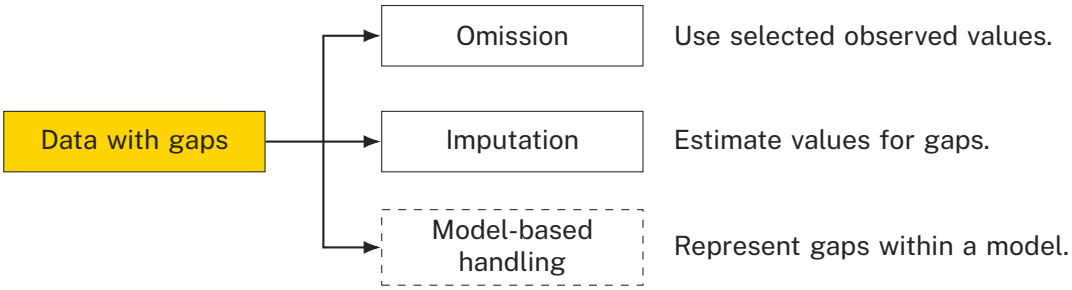


Figure 2. Three ways to proceed when values are missing

The rule Complete-case deletion uses only rows observed on every variable the analysis needs. Available-case summaries use every observed value each summary needs. A two-variable statistic uses complete pairs, which is why the method is also called pairwise deletion. Different summaries may then describe different subsets, so always report their counts. Table 7 sets out the trade-offs.

Table 7. Complete-case and available-case trade-offs

Complete-case deletion	Available-case summaries
Every required variable is observed for each retained row.	Each summary uses the values observed for its own variables.
The summaries describe one common subset.	Different summaries may use different subsets and counts.
Can discard otherwise usable values.	Comparisons require care about which rows contributed.

If the sampling process supports an unbiased mean, MCAR missingness keeps that property, but fewer observations still carry less information. MCAR cannot correct bias already present in the sampling. When missingness is systematic, the observed subset may differ from the population of interest.

Worked example In the grades table, requiring both scores keeps Luis and Jose. Their means are Math $(90+70)/2 = 80$ and English $(86+60)/2 = 73$. Using every available grade gives Math $245/3 \approx 81.67$ from three values and English $146/2 = 73$ from two. The counts must accompany the means.

R

```r
scores <- clean_grades[c("Math", "English")]
complete <- scores[complete.cases(scores), ]
nrow(complete) # 2
colMeans(complete)
# Math 80, English 73
colMeans(scores, na.rm = TRUE)
# Math 81.66667, English 73
colSums(!is.na(scores))
# Math 3, English 2
```

Python

```python
scores = clean_grades[["Math", "English"]]
complete = scores.dropna()
print(len(complete)) # 2
print(complete.mean())
# Math 80.0, English 73.0
print(scores.mean())
# Math 81.666667, English 73.000000
print(scores.notna().sum())
# Math 3, English 2
```

Differences to know

- R's `colMeans` needs `na.rm=TRUE` to skip missing values; pandas `mean` skips them by default. Neither default decides whether omission is defensible.
- Correlation differs the same way. R's `cor` needs an explicit missing-value rule, while pandas uses available pairs by default. When different pairs use different rows, a set of correlations need not describe one common sample.

6 Filling gaps with a central value

What it is for Imputation replaces each gap with an estimate so that every row can be used. The estimate is not the lost value, so keep a record of which entries were originally missing.

The rule Table 8 lists the strategies and the information each uses. A central-value substitution uses a mean, median or mode, and these are not interchangeable. The mean and median apply to numeric variables. The mode can apply to categorical variables, and a mode rule must state how ties between equally frequent categories are broken.

Table 8. Imputation strategies and the information each uses

Central value	A mean, median, or mode, selected for the variable type.
Ordered measurements	Previous value (LOCF), baseline value (BOCF), or interpolation between observations. Requires a justified time-series assumption.
Prediction	Other recorded variables inform a predicted numeric value or category.
Multiple imputation	Several plausible completed datasets and a combined analysis that represents uncertainty.

Worked example, mean or median fill Take the eight observations $2, 4, 4, 5, 7, 9, 12, 41$ and add two gaps, kept in `with_gaps`. Fill separate copies. The observed mean is $84/8 = 10.5$ and the observed median is $(5 + 7)/2 = 6$. Mean fill keeps the mean at $105/10 = 10.5$. Median fill changes it to $(84 + 6 + 6)/10 = 9.6$. The effect of one substitution rule cannot be claimed for all central-value rules.

R

```r
observed <- c(2, 4, 4, 5, 7, 9, 12, 41)
with_gaps <- c(observed, NA, NA)
mean_filled <- with_gaps
mean_filled[is.na(mean_filled)] <-
  mean(observed)
median_filled <- with_gaps
median_filled[is.na(median_filled)] <-
  median(observed)
c(mean(mean_filled), sd(mean_filled))
# 10.5, 11.22497
c(mean(median_filled), sd(median_filled))
# 9.6, 11.3842
```

Python

```python
observed = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
with_gaps = pd.concat(
    [observed, pd.Series([np.nan, np.nan])],
    ignore_index=True)
mean_filled = with_gaps.fillna(observed.mean())
median_filled = with_gaps.fillna(
    observed.median())
print(mean_filled.mean(), mean_filled.std())
# 10.5 11.224972160321824
print(median_filled.mean(), median_filled.std())
# 9.6 11.384199576606166
```

Worked example, what mean fill does to spread Each mean-filled gap adds a zero deviation while increasing the count. The sum of squared deviations stays at 1134, but the divisor grows from 7 to 9.

Sample variance before and after two mean-filled gaps

Observed values ($n = 8$)	Two mean-filled gaps ($n = 10$)
$\bar{x} = 84/8 = 10.5$	$\bar{x} = 105/10 = 10.5$
$\sum_i (x_i - \bar{x})^2 = 1134$	$1134 + 0 + 0 = 1134$
$s_x^2 = 1134/7 = 162$	$s_x^2 = 1134/9 = 126$
$s_x = \sqrt{162} \approx 12.73$	$s_x = \sqrt{126} \approx 11.22$

The standard deviation falls from about 12.73 to 11.22, a reduction of about 11.8% relative to the observed-data SD. The true complete-data SD is unknown because the two original values are missing. Figure 3 shows the two filled values sitting exactly at the mean.

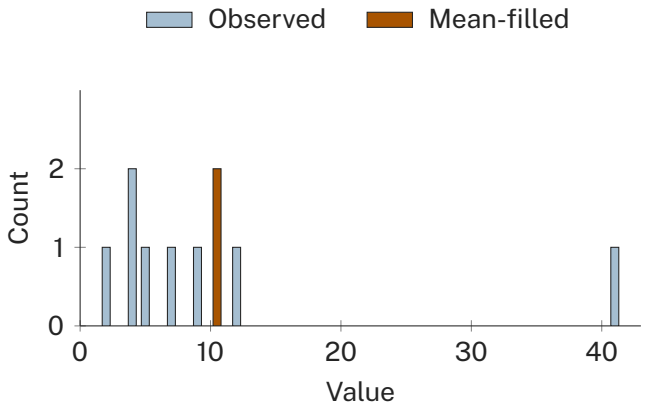


Figure 3. The eight observed values and two additional values imputed at the mean

What mean fill does and does not do

- It keeps the observed mean exactly, under any missingness mechanism. That is arithmetic. Whether the observed mean estimates the population mean without selection bias is a separate question, which MCAR answers.
- It reduces the sample variance by adding zero deviations.
- It can distort relationships with other variables.
- Treating filled values as observed understates uncertainty. It does not restore the lost information or remove selection bias.

7 Filling gaps in ordered measurements

What it is for When one subject is measured repeatedly over time, neighbouring measurements carry information about a gap.

The rules Last observation carried forward (LOCF) uses the most recent observed value. Baseline observation carried forward (BOCF) uses the first, baseline value. Backward fill uses a *later* observed value and is a different operation. Interpolation estimates values between observations. It is one way to impute interior gaps, not a name for all imputation.

Before applying any of these, sort by time within each subject, check the spacing, and choose a rule for gaps at either end. Do not carry values between subjects or across an unjustified time gap. Baseline fill needs a known baseline. Interpolation needs observed values on both sides and a reasonable assumption about change between them.

Worked example One subject is measured at five equally spaced, ordered times. The series is 10, missing, 14, missing, 18, with no endpoint gaps.

- Forward fill gives $10, 10, 14, 14, 18$.
- Baseline fill gives $10, 10, 14, 10, 18$.
- Straight-line interpolation gives $10, 12, 14, 16, 18$.

None is justified merely because it is easy to compute. Both languages print these three series.

R

```r
series <- c(10, NA, 14, NA, 18)
forward <- tidyr::fill(data.frame(series),
                       series)$series
baseline <- series
baseline[is.na(baseline)] <- series[1]
inside <- approx(seq_along(series), series,
                 xout = seq_along(series))$y
forward   # 10 10 14 14 18
baseline  # 10 10 14 10 18
inside    # 10 12 14 16 18
```

Python

```python
series = pd.Series([10, np.nan, 14, np.nan, 18])
forward = series.ffill()
baseline = series.fillna(series.iloc[0])
inside = series.interpolate(
    method="linear", limit_area="inside")
print(forward.tolist())
# [10.0, 10.0, 14.0, 14.0, 18.0]
print(baseline.tolist())
# [10.0, 10.0, 14.0, 10.0, 18.0]
print(inside.tolist())
# [10.0, 12.0, 14.0, 16.0, 18.0]
```

8 Standardizing numeric variables

What it is for With units, types and missingness decisions documented, the next step is a representation that suits the analysis. Standardization expresses each value as a distance from the mean in standard-deviation units, so variables measured in different units can be compared.

The formula For a numeric variable with at least two observed values and positive sample SD,

$$z_i = \frac{x_i - \bar{x}}{s_x}.$$

The transformed values have mean 0 and sample SD 1, apart from rounding. Shape and ordering do not change. A constant column has zero SD and needs a separate documented decision instead of division by zero. Missing values stay missing unless an earlier justified step handled them.

Worked example For $x = 2, 4, 4, 5, 7, 9, 12, 41$, $\bar{x} = 10.5$ and $s_x = \sqrt{1134/7} = \sqrt{162} \approx 12.73$. Then $z_1 = -8.5/12.73 \approx -0.67$, $z_4 = -5.5/12.73 \approx -0.43$ and $z_8 = 30.5/12.73 \approx 2.40$. The value 41 is still unusual. Changing units does not repair a data problem.

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
stopifnot(length(x) > 1, sd(x) > 0)
z <- (x - mean(x)) / sd(x)
z[1] # -0.6678231
mean(z) # approximately 0
sd(z) # 1
as.vector(scale(x))[1] # -0.6678231
```

Python

```python
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41], dtype=float)
assert len(x) > 1 and x.std(ddof=1) > 0
z = (x - x.mean()) / x.std(ddof=1)
print(z.iloc[0]) # -0.6678230711206282
print(z.mean()) # approximately 0
print(z.std(ddof=1)) # 1.0
# Both examples use the sample SD.
```

Differences to know

- R's `sd()` and `scale()` use divisor $n - 1$, and so does pandas `.std()`.
- NumPy's `np.std` and scikit-learn's `StandardScaler` use divisor n. For these eight values that divisor gives $s = (1134/8)^{1/2} \approx 11.91$, so the first standardized value is about -0.713933 instead of -0.667823. The Python code states `ddof=1` explicitly. State the convention you use.

9 Discretizing numeric variables

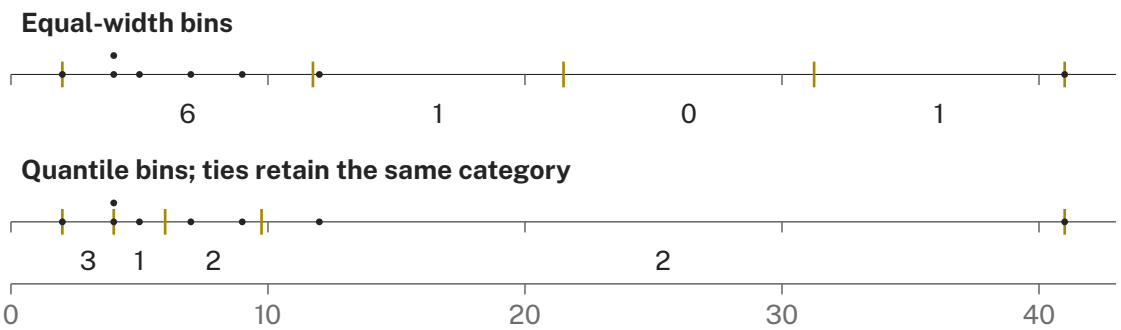
What it is for Discretization replaces numeric values with categories. Some methods need categorical input, and a few categories can simplify communication or computation. It also loses numerical detail, so choose boundaries for the question.

The rules Equal-width bins divide a chosen range into k intervals of equal width. Quantile bins aim for similar counts in each bin. Ties can prevent equal counts, because equal values are never split across categories to balance them. Here k is the number of bins and n is still the number of observations.

Worked example Use $x = 2, 4, 4, 5, 7, 9, 12, 41$ with $k = 4$.

- Equal width, $(41 - 2)/4 = 9.75$, gives edges $2, 11.75, 21.5, 31.25, 41$. The counts are 6,1,0,1.
- Quantile edges are the 0, 25, 50, 75 and 100th percentiles, $2, 4, 6, 9.75, 41$. The counts are 3,1,2,2, because the first bin includes both observations equal to 4.

The two marks at 4 are stacked vertically in Figure 4; neither value is shifted. The observation at 41 leaves several equal-width bins sparse. That can be useful for showing the tail, but not for forming groups of similar size.



Numbers beneath intervals are counts; both axes share the value scale.

Figure 4. The same eight observations under two four-bin rules

R

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
k <- 4
width_edges <- seq(min(x), max(x),
                   length.out = k + 1)
quantile_edges <- quantile(
  x, probs = seq(0, 1, 1/k))
table(cut(x, width_edges,
          include.lowest = TRUE))
# 6, 1, 0, 1
table(cut(x, quantile_edges,
          include.lowest = TRUE))
# 3, 1, 2, 2
```

Python

```python
x = pd.Series([2, 4, 4, 5, 7, 9, 12, 41])
k = 4
width_edges = np.linspace(
    x.min(), x.max(), k + 1)
quantile_edges = x.quantile(
    np.linspace(0, 1, k + 1))
print(pd.cut(
    x, width_edges, include_lowest=True
).value_counts(sort=False))
# 6, 1, 0, 1
print(pd.cut(
    x, quantile_edges, include_lowest=True
).value_counts(sort=False))
# 3, 1, 2, 2
```

Both versions include the minimum explicitly. Save the edges and reuse them when categories must be comparable across data sets. Re-estimating the minimum, maximum or quantiles on each data set can change what a category means. If quantile edges repeat, use fewer bins and document the choice. Do not silently move tied observations.

Differences to know

- R's `include.lowest=TRUE` and pandas `include_lowest=True` put the minimum in the first bin.
- R's `cut(x, breaks=k)` and pandas `cut(x, k)` compute edges automatically and slightly extend the range to include its endpoints. Their printed labels can round differently.
- pandas `qcut` makes quantile bins directly and raises an error for repeated edges unless told to drop duplicates. Saved edges give the same reuse without a modelling pipeline.

10 Worked case, age non-disclosure

What it is for This case combines the missingness mechanisms with binning and a missingness record.

The setup A survey records age and consumption. Respondents under 21 are less likely to disclose their age, even at the same consumption level. No other variables are recorded.

The mechanism Missingness depends on the unobserved age itself, after accounting for recorded consumption. This suggests MNAR relative to the recorded information. Additional observations or a different collection process could change that assessment. The simple methods in this class give no general guarantee of removing the resulting selection bias.

What binning and a flag can do Age bins summarize observed ages, but they cannot recover ages that were never reported. A refusal does not show that the respondent is under 21, so keep unknown ages missing. A separate nominal "age not disclosed" flag records the missingness event without inventing an age. It helps describe the recording process, but it does not resolve MNAR bias or recover the ages. Do not merge "under 21" and "not disclosed" into a single age level.

11 Choosing a missing-data strategy

What it is for Every earlier section ends in a decision that must be written down. This is the procedure for making it.

The rule Given missing measurements, work through these steps.

1. State the question and the available information.
2. Give a plausible explanation for the gaps, and name the mechanism it implies.
3. Report observed values and their counts when that answers the question.
4. If filling a gap is defensible, name the rule, keep the missingness record, and state the limitation the rule creates.
5. Seek additional information when the decision is unsupported.

12 Summary and Lab 02 connection

Define the observational unit before reshaping. Use the data dictionary to interpret units, types and missing codes. Report counts with summaries and justify deletion or imputation. Standardization changes scale; binning reduces detail. Keep the raw data and record every transformation so another person can follow the reasoning.

Lab 02 begins next class and builds on the Lab 01 workflow. Bring your record of the decisions you made in Lab 01. Lab 02 uses a new data set, chosen next class from the Lab 02 catalogue, so the Lab 01 data set does not carry over. Work in R or Python, locally or in an approved cloud environment. No installation of the other language is required.

Lab 02 extends Lab 01 with inspection, cleaning, documented decisions and before/after summaries. Follow its handout for the required operations and submission details. It does not require imputation models or other model fitting. Keep assessed code, outputs and reports private until the deadline and instructor release. Commit corrections as new commits, and preserve the submitted snapshot and its history.

References

[1] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth. *CRISP-DM 1.0: Step-by-step Data Mining Guide*. Tech. rep. The CRISP-DM Consortium, 2000.

[2] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. 3rd ed. Wiley, 2019.

[3] D. B. Rubin. “Inference and missing data”. In: *Biometrika* 63.3 (1976), pp. 581–592.

[4] H. Wickham. “Tidy data”. In: *Journal of Statistical Software* 59.10 (2014), pp. 1–23.